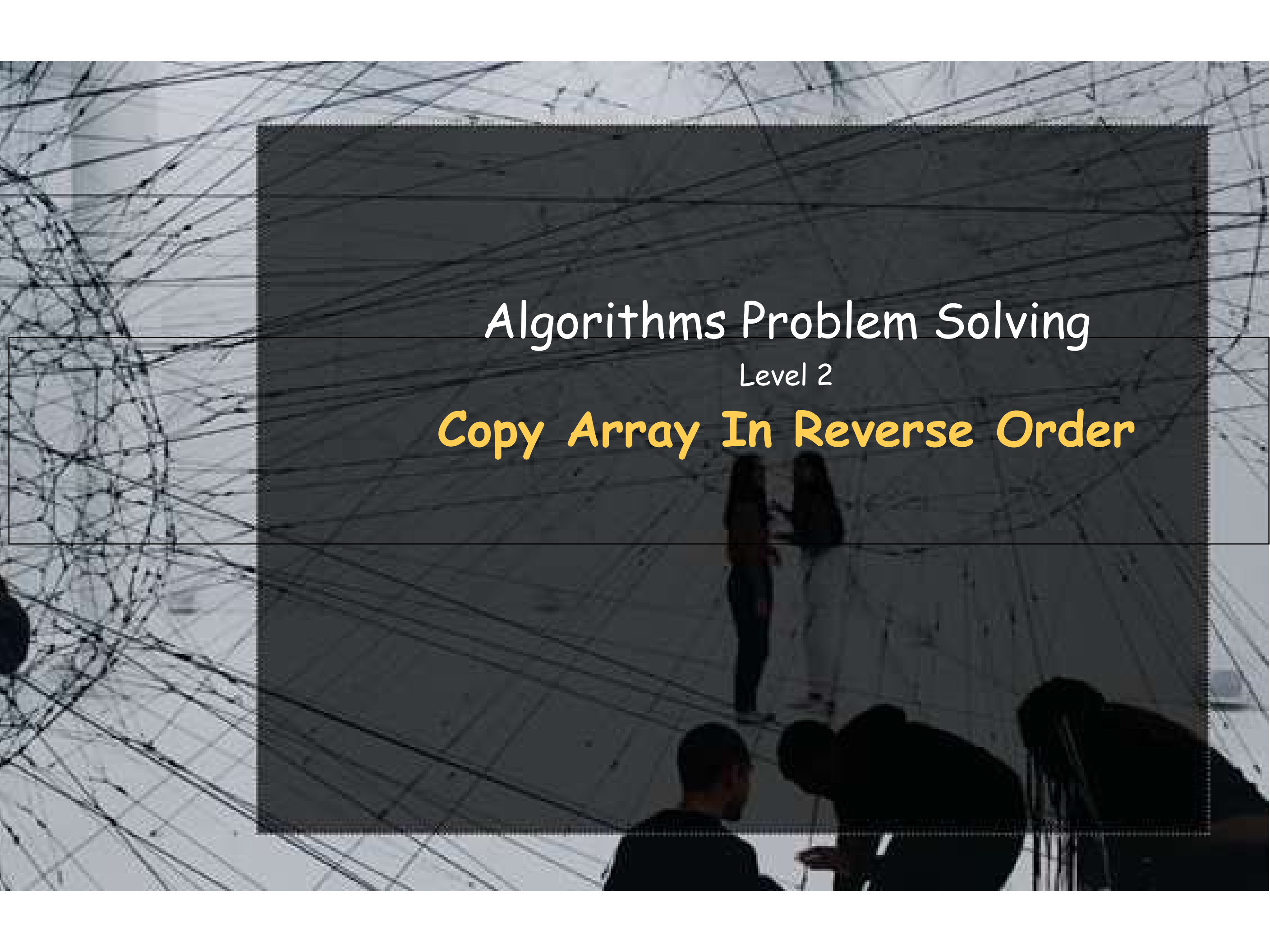


Algorithms Problem Solving

Level 2

Copy Array In Reverse Order



Algorithms Problem Solving

Level 2

Copy Array In Reverse Order

Definition

- ◆ This program fills an array with random numbers from `1` to `100`.
- ◆ It copies the elements into a second array in reverse order.
- ◆ The first element of `arr1` becomes the last element of `arr2`.
- ◆ The last element of `arr1` becomes the first element of `arr2`.

Explanation

- ◆ `FillArrayWithRandomNumber()` fills `arr1` with random numbers from `1` to `100`.
- ◆ `RverseArray()` copies the elements from `arr1` to `reverseArr` in reverse order.
- ◆ The important statement is:
`reverseArr[i] = arr[arrLength - 1 - i];`
- ◆ `arrLength - 1` represents the last valid index of the array.
- ◆ As `i` increases, the source index moves backward.
- ◆ For an array with `10` elements:
 - `i = 0 → arr[9]`
 - `i = 1 → arr[8]`
 - `i = 2 → arr[7]`
 - ...
 - `i = 9 → arr[0]`

Steps in main()

1. Ask the user for the number of elements.
2. Fill `arr1` with random numbers from `1` to `100`.
3. Print `arr1`.
4. Loop through all elements of `arr1`.
5. Access the elements starting from the last element.
6. Copy each element into the corresponding position in `arr2`.
7. Print `arr2`.

💡 Example

◆ `Array 1`:

2 95 66 41 19 28 35 85 7 41

◆ The elements are copied from right to left:

$\text{arr2}[0] = \text{arr1}[9] \rightarrow 41$

$\text{arr2}[1] = \text{arr1}[8] \rightarrow 7$

$\text{arr2}[2] = \text{arr1}[7] \rightarrow 85$

$\text{arr2}[3] = \text{arr1}[6] \rightarrow 35$

...

◆ Therefore:

41 7 85 35 28 19 41 66 95 2

➡ Result

Array 1 Elements: 2 95 66 41 19 28 35 85 7 41

Array 2 Elements after copying array2 in reverse: 41 7 85 35 28 19 41 66 95 2

✈ Note

- ◆ The array indexes start from `0`.
- ◆ If the array length is `10`, the indexes are:
0 1 2 3 4 5 6 7 8 9
- ◆ Therefore, the last index is always `arrLength - 1`.
- ◆ The expression:
$$\text{arrLength} - 1 - i$$
- ◆ is what makes the copying happen in reverse order.
- ◆ `arr1` is not changed. Only `arr2` receives the elements in reverse order.
- ◆ The function name `RverseArray` has a spelling mistake; it should ideally be `ReverseArray`.

⊘ Warning

- ◆ ``arrayLength`` should not exceed ``100``.
- ◆ ``ReadArrayElements()``, ``FillArrayWithNumbersFrom1ToN()``, ``Swap()``, and ``ShuffleArray()`` are defined but not used in this program.
- ◆ ``cmath`` and ``corecrt_math_defines.h`` are unnecessary.
- ◆ ``rand()`` is pseudo-random and is not suitable for security-sensitive applications.
- ◆ Since ``srand((unsigned)time(NULL))`` is used, the appropriate random/time headers should ideally be included explicitly.



Comparison

▨ `arr1`

- ✓ Contains the original elements.
- ✓ The order remains unchanged.
- ✓ Example: `2 95 66 41 19`.

▤ `arr2`

- ✓ Contains the elements of `arr1` in reverse order.
- ✓ The last element of `arr1` becomes the first element of `arr2`.
- ✓ Example: `2 95 66 41 19` → `19 41 66 95 2`.

✦ Summary

- ✓ Fill the first array with random numbers.
- ✓ Create a second array for the reversed copy.
- ✓ Start reading the first array from its last index.
- ✓ Use ``arrLength - 1 - i`` to access elements in reverse.
- ✓ Store the reversed elements in ``arr2``.
- ✓ The original array remains unchanged.

Thank you 😊